

CCES – College of Computer Engineering and Sciences

CS3401 – Algorithm Design and Analysis

Chapter 2- Algorithm analysis and mathematical background

References:

- ❖ *Introduction to the Design and Analysis of Algorithms. Anany V. Levitin, Pearson 3rd Edition.*
- ❖ *Data Structures and Algorithm Analysis in JAVA, M. A. Weiss. Pearson 2014.*

Algorithm analysis

Objectives

We shall discuss:

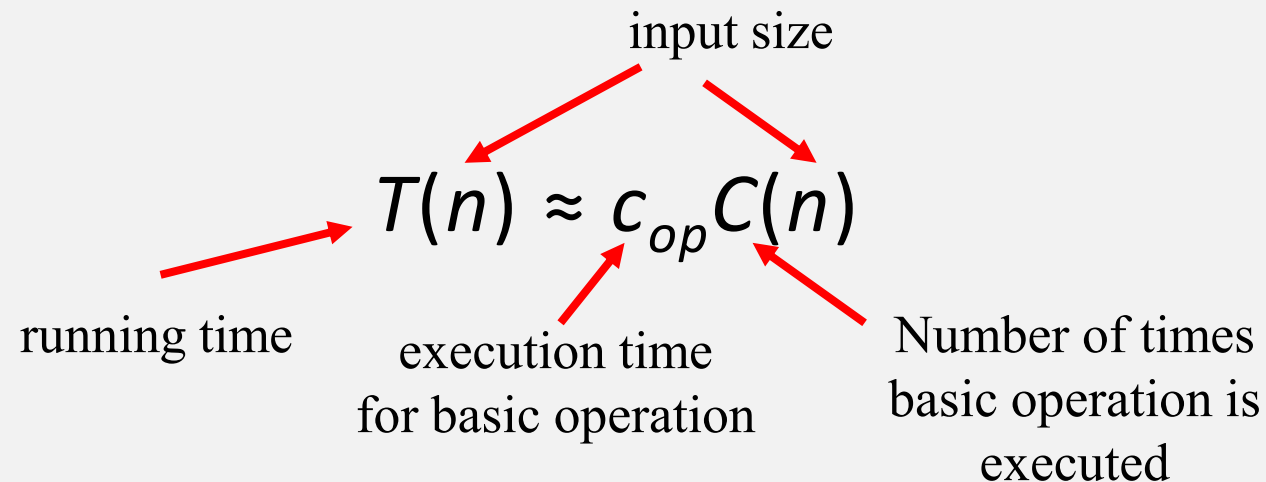
- How to estimate the time required for a program.
- How to reduce the running time of a program from days or years to fractions of a second.
- The results of careless use of recursion.

Algorithm analysis

Theoretical analysis of time efficiency

Time efficiency is analyzed by determining the number of repetitions of the basic operation as a function of input size

- Basic operation: the operation that contributes most towards the running time of the algorithm



Algorithm analysis

Input size and basic operation examples

Problem	Input size measure	Basic operation
Searching for key in a list of n items	Number of list's items, i.e. n	Key comparison
Multiplication of two matrices	Matrix dimensions or total number of elements	Multiplication of two numbers
Checking primality of a given integer n	n 's size = number of digits (in binary representation)	Division
Typical graph problem Travelling Salesman Problem	#vertices and/or edges	Visiting a vertex or traversing an edge

Algorithm analysis

Worst Case, Best Case, and Average Case

For some algorithms efficiency depends on form of input:

1. **Worst Case:** $C_{\text{worst}}(n)$ – maximum over inputs of size n .
2. **Best Case:** $C_{\text{best}}(n)$ – minimum over inputs of size n .
3. **Average Case:** $C_{\text{avg}}(n)$ – “average” over inputs of size n .
 - Number of times the basic operation will be executed on typical input
 - NOT the average of worst and best case
 - Expected number of basic operations considered as a random variable under some assumption about the probability distribution of all possible inputs

Algorithm analysis

Worst Case, Best Case, and Average Case

❖ Best case : Searching key element present at first index

❑ Best Case Time : 1 $O(1)$, $B(n) = O(1)$

❖ Worst Case: Searching a key element, present at last index

❑ Worst Case Time= n , $O(n)$, $W(n) = O(n)$

❖ Average case: (all possible case time) / no. of cases

❑ Avg. Time = $(1+2+3+4+\dots +n)/n = ((n(n+1))/2)/n = (n+1)/2$

ALGORITHM *SequentialSearch*($A[0..n - 1]$, K)

//Searches for a given value in a given array by sequential search

//Input: An array $A[0..n - 1]$ and a search key K

//Output: The index of the first element in A that matches K

// or -1 if there are no matching elements

$i \leftarrow 0$

while $i < n$ **and** $A[i] = K$ **do**

$i \leftarrow i + 1$

if $i < n$ **return** i

else return -1

Asymptotic growth

The rate of growth of a function

Given a particular differentiable function $f(N)$, all other differentiable functions fall into three classes:

- growing with the same rate
- growing faster
- growing slower

Classifying Functions by Their Asymptotic Growth:

- Little oh (o), Little omega (ω)
- Big theta (Θ)
- Big Oh (O), Big Omega (Ω)

The “Big-Oh” Notation

$T(N) = O(f(N))$ if there are positive constants c and n_0 such that

$$T(N) \leq c f(N) \text{ when } N \geq n_0$$

$$T(N) = O(f(N)) \quad \text{if} \quad \exists c, n_0 \in R \quad s.t.$$

$$T(N) \leq cf(N) \quad \text{when} \quad N \geq n_0$$

- We read $T(N) = O(f(N))$ as $f(N)$ is big-Oh of $T(N)$
- The big-Oh notation gives an **upper bound** on the growth rate of a function.
- The statement “ $T(N)$ is $O(f(N))$ ” means that the growth rate of $T(N)$ is no more than the growth rate of $f(N)$.

Algorithm analysis

Mathematical Background

Rules

If $T_1(N) = O(f(N))$ and $T_2(N) = O(g(N))$, then

(a) $T_1(N) + T_2(N) = \max(O(f(N)), O(g(N)))$

(b) $T_1(N) \times T_2(N) = O(f(N) \times g(N))$

Examples

If $T_1(N) = O(N^2)$ and $T_2(N) = O(N)$ then

(a) $T_1(N) + T_2(N) = O(N^2)$

(b) $T_1(N) * T_2(N) = O(N^3)$



Do not say $T(N) = O(2N^2)$ or $T(N) = O(N^2 + N)$. The correct form is $T(N) = O(N^2)$

Algorithm analysis

Mathematical Background

Function	Name
c	Constant
$\log N$	Logarithmic
$\log^2 N$	Log-squared
N	Linear
$N \log N$	Linearithmic
N^2	Quadratic
N^3	Cubic
2^N	Exponential

Typical Growth Rates

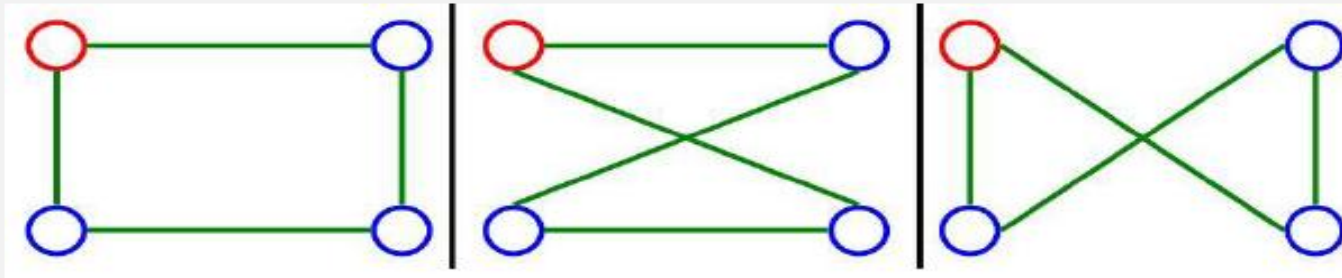
The functions are arranged by the most of growth rate

Algorithm analysis

Mathematical Background

Example of an exponential algorithm

Find the shortest path between 4 points in a complete graph. The algorithm enumerate all possibilities. For $n = 4$ there are 3 possibilities. For each possibility route, the algorithm compute its length and in the end it return the shortest one.



For $n = 5$ there are 12 possibilities.

For $n = 4$ there are 60 possibilities.

In general, the number of possibilities is equal to $\frac{(n-1)!}{2}$ where n is the number of points in the graph.

Algorithm analysis

Mathematical Background

Example of an exponential algorithm

If the computer when we run the algorithm take 10^{-6} second to evaluate one possibility we can draw the following table.

n	Number of possible routes	Computational time
4	3	3 μ s
5	12	12 μ s
10	$1.81 \cdot 10^5$	0.18 second
15	$4.36 \cdot 10^{10}$	12 hours
20	$6.08 \cdot 10^{16}$	1928 years
25	$3.10 \cdot 10^{23}$	9.8 milliard years

Combinatorial explosion !

Algorithm analysis

The “Big-Omega” Notation

$$T(N) = \Omega(g(N)) \quad \text{if} \quad \exists c, n_0 \in R \quad \text{s.t.} \\ T(N) \geq cg(N) \quad \text{when} \quad N \geq n_0$$

- The big-Omega notation gives an **lower bound** on the growth rate of a function.
- The statement “ $T(N)$ is $\Omega(g(N))$ ” means that the growth rate of $T(N)$ is greater than or equal the growth rate of $g(N)$.
- For example, $N^4 = \Omega(N^3)$

Algorithm analysis

The “Big-Theta” Notation

$$T(N) = \Theta(h(N)) \quad \text{iff} \quad T(N) = O(h(N)) \text{ and } T(N) = \Omega(h(N))$$

- The statement “ $T(N)$ is $\Theta(h(N))$ ” means that the growth rate of $T(N)$ is equals the growth rate of $h(N)$.

- **Important:** If $T(N)$ is a polynomial of degree k , then

$$T(N) = \Theta(N^k)$$

- For example,

$$2N^4 = \Theta(N^4)$$

Algorithm analysis

The “Little-oh” Notation

$T(N) = o(\rho(N))$ if for all constants c there exists an n_0 such that
 $T(N) < c \rho(N)$ when $N \geq n_0$

$T(N) = o(\rho(N))$ if
 $T(N) = O(\rho(N))$ and $T(N) \neq \Theta(\rho(N))$

- The statement “ $T(N)$ is $o(\rho(N))$ ” means that the growth rate of $T(N)$ is less than the growth rate of $\rho(N)$.
- For example,

$$N^3 = o(N^4)$$

Algorithm Analysis

An Example

- Use the most appropriate notation among O , ω , and θ to indicate the time efficiency of $50x^2 + 7$. In terms of :
 - a) Worst case: ??
 - b) Best case: ??
 - c) Average case: ??

Algorithm analysis

Establishing order of growth using limits

$$\lim_{n \rightarrow \infty} T(n)/g(n) = \begin{cases} 0 & \text{order of growth of } T(n) < \text{order of growth of } g(n) \\ c > 0 & \text{order of growth of } T(n) = \text{order of growth of } g(n) \\ \infty & \text{order of growth of } T(n) > \text{order of growth of } g(n) \end{cases}$$

Examples:

✓ $10n$ vs n^2

$$\lim_{n \rightarrow \infty} \frac{10n}{n^2} = \lim_{n \rightarrow \infty} \frac{10}{n} = 0 \Rightarrow 10n = O(n^2)$$

✓ $\frac{n(n-1)}{2}$ vs n^2

$$\lim_{n \rightarrow \infty} \frac{n(n-1)}{2n^2} = \frac{1}{2} \lim_{n \rightarrow \infty} \frac{n-1}{n} = \frac{1}{2} \lim_{n \rightarrow \infty} \left(1 - \frac{1}{n}\right) = \frac{1}{2} (1 - 0) = \frac{1}{2} \Rightarrow \frac{n(n-1)}{2} = \Theta(n^2)$$

Algorithm analysis

What to Analyze?

- Factors affecting the running time:
 - Computer,
 - Compiler,
 - Algorithm used, and
 - Input to the algorithm
 - The content of the input affects the running time
 - The input size is the main consideration (e.g. sorting problem $\Rightarrow$ the number of items to be sorted)

Obviously beyond the scope of any theoretical model, so, although they are important, we cannot deal with them here.

The Maximum Subsequence Sum Problem

Given (possibly negative) integers $A_1, A_2, \dots, A_N$, find the maximum value of

$$\sum_{k=i}^j A_k$$

For convenience, the maximum subsequence sum is zero if all the integers are negative.

- Example: input -2, 11, -4, 13, -5, -2
- the answer is 20 (A_2 through A_4)

Algorithm analysis

What to Analyze?

Input Size	Algorithm Time			
	1 $O(N^3)$	2 $O(N^2)$	3 $O(N \log N)$	4 $O(N)$
$N=100$	0.000159	0.000006	0.000005	0.000002
$N=1000$	0.095857	0.000371	0.000060	0.000022
$N=10,000$	86.67	0.033322	0.000619	0.000222
$N=100,000$	NA	3.33	0.006700	0.002205
$N=1,000,000$	NA	NA	0.074870	0.022711

Running times of several algorithms for maximum subsequence sum
(in seconds)

Algorithm analysis

Running Time Calculation

- Here is a simple program fragment to calculate

$$\sum_{i=1}^N i^3$$

```
int sum (int n)
{
    int i, partialSum;
    partialSum = 0;
    for ( i = 1; i <= n; i++ )
        partialSum += i * i * i;
    return partialSum;
}
```

We ignore the costs of calling the function, thus the total is 1 to initialize N

The declarations count for no time.

The cost is 1 to initialize Sum.

The total cost for this line is 1 to initialize i , $N+1$ for all the tests ($i \leq N$), and N for all the increments, which is $2N + 2$.

We ignore the costs of returning.

This line counts for four units per time executed and is executed N times, for a total of $4N$ units.

- Thus the total cost is $6N + 4$ which is equal $O(N)$.

Rule 1 the for loop

- The running time of a for loop is at most [the running time of the statements inside the for loop (including tests)] x [the number of iterations].
- For example:

```
for ( i = 1; i <= n ; i++ )  
    k++;
```
- Total cost is $1 + N + 1 + N + N * 1 = 3N + 2$, which has $O(N)$.

Rule 2 Nested for loops

- The total running time of a statement inside a group of nested loops is the running time of the statement multiplied by the product of the sizes of all the for loops.

- For example:

```
for ( i = 0; i < n; i++ )  
    for ( j = 0; j < n; j++ )  
        k++
```

- Total cost is $1 + (N+1) + N + N * (1 + (N+1) + N + N * 1) = 3N^2 + 4N + 2$, which has $O(N^2)$.

Algorithm analysis

General Rules

$O(N^2)$ Program Fragment

```
sum = 0
for ( j = 0; j < n; j++ )
    for ( k = 0; k < j; k++ )
        sum++;
```

$$\sum_{j=0}^{N-1} \sum_{k=0}^{j-1} 1 = \sum_{j=0}^{N-1} j = \sum_{j=1}^{N-1} j = \frac{(N-1)N}{2}$$

$O(N^3)$ Program Fragment

```
sum = 0
for ( j = 0; j < n; j++ )
    for ( k = 0; k < n * n; k++ )
        sum++;
```

$$\sum_{j=0}^{N-1} \sum_{k=0}^{N^2-1} 1 = \sum_{j=0}^{N-1} N^2 = N^2 N = N^3$$

Rule 3 Consecutive Statements

- For example:

```
for (i = 0; i < n; i++)  
    a[ i ] = 0;  
for (i = 0; i < n; i++)  
    for (j = 0; j < n; j++)  
        a[ i ] += a[ j ] + i + j;
```

- The program fragment, which has $O(N)$ work followed by $O(N^2)$ work, is also $O(N^2)$.

Rule 4 Condition Statement

```
if ( condition )  
    S1  
else  
    S2
```

- The running time of a if/else statement is never more the running time of the test plus the larger of the running times of S1 and S2.

Algorithm analysis

Useful summation formulas and rules

$$\sum_{i=a}^b \text{constant} = \text{constant} \times (b - a + 1)$$

$$\sum_{i=0}^N i = \sum_{i=1}^N i = \frac{N^2 + N}{2}$$

$$\sum_{i=0}^{N-1} i = \sum_{i=1}^{N-1} i = \frac{N^2 - N}{2}$$

$$\sum_{i=0}^{N+1} i = \sum_{i=1}^{N+1} i = \frac{N^2 + 3N + 2}{2}$$

$$\sum_{i=0}^N i^2 = \sum_{i=1}^N i^2 = \frac{2N^3 + 3N^2 + N}{6}$$

$$\sum_{i=1}^N 2^i = 2^{N+1} - 1$$

Algorithm analysis

Logarithms in the Running Time

- An algorithm is $O(\log N)$ if it takes constant ($O(1)$) time to cut the problem size by a fraction (which is usually 0.5).
- Anything is halved on each iteration, then you usually get $O(\log N)$.
- If you have an algorithm which runs in $O(\log N)$ time, take it because it will be very fast.

Exponentiation

The following algorithm for computing X^N where X and N are two positive integers.

$$X^N = \begin{cases} 1 & \text{if } N = 0 \\ X & \text{if } N = 1 \\ X^{N/2} \cdot X^{N/2} & \text{if } N \text{ is even} \\ X^{(N-1)/2} \cdot X^{(N-1)/2} \cdot X & \text{if } N \text{ is odd} \end{cases}$$

Algorithm analysis

Logarithms in the Running Time

Exponentiation

```
long pow( long x, int n )
{
    if( n == 0 )
        return 1;
    if( n == 1 )
        return x;
    if( isEven( n ) )
        return pow( x * x, n / 2 );
    else
        return pow( x * x, n-1 / 2 ) * x;
}
```

$$\begin{aligned} T(N) &= T\left(\frac{N}{2}\right) + c \\ &= T\left(\frac{N}{2^2}\right) + 2c \\ &= T\left(\frac{N}{2^3}\right) + 3c \\ &\vdots \\ &= T\left(\frac{N}{2^k}\right) + kc \end{aligned}$$

Running Time:
 $O(\log N)$

Algorithm analysis

Analysis of Recursive Algorithms

Example 1: Recursive evaluation of $n!$

Definition: $n! = 1 \cdot 2 \cdot \dots \cdot (n-1) \cdot n$ for $n \geq 1$ and $0! = 1$

Recursive definition of $n!$: $F(n) = F(n-1) \cdot n$ for $n \geq 1$ and
 $F(0) = 1$

The number of multiplications made

$$M(n) = \underbrace{M(n-1)}_{\text{to compute } F(n-1)} + \underbrace{1}_{\text{to multiply } F(n-1) \text{ by } n} \quad \text{for } n > 0.$$

$$\begin{aligned} M(n) &= M(n-1) + 1 \\ &= [M(n-2) + 1] + 1 = M(n-2) + 2 \\ &= [M(n-3) + 1] + 2 = M(n-3) + 3. \end{aligned}$$

$$M(n) = M(n-1) + 1 = \dots = M(n-i) + i = \dots = M(n-n) + n = n.$$

ALGORITHM $F(n)$

```
//Computes  $n!$  recursively  
//Input: A nonnegative integer  $n$   
//Output: The value of  $n!$   
if  $n = 0$  return 1  
else return  $F(n-1) * n$ 
```

Algorithm analysis

Analysis of Recursive Algorithms

Example 2: Counting #bits

ALGORITHM *BinRec*(n)

//Input: A positive decimal integer n

//Output: The number of binary digits in n 's binary representation

if $n = 1$ **return** 1

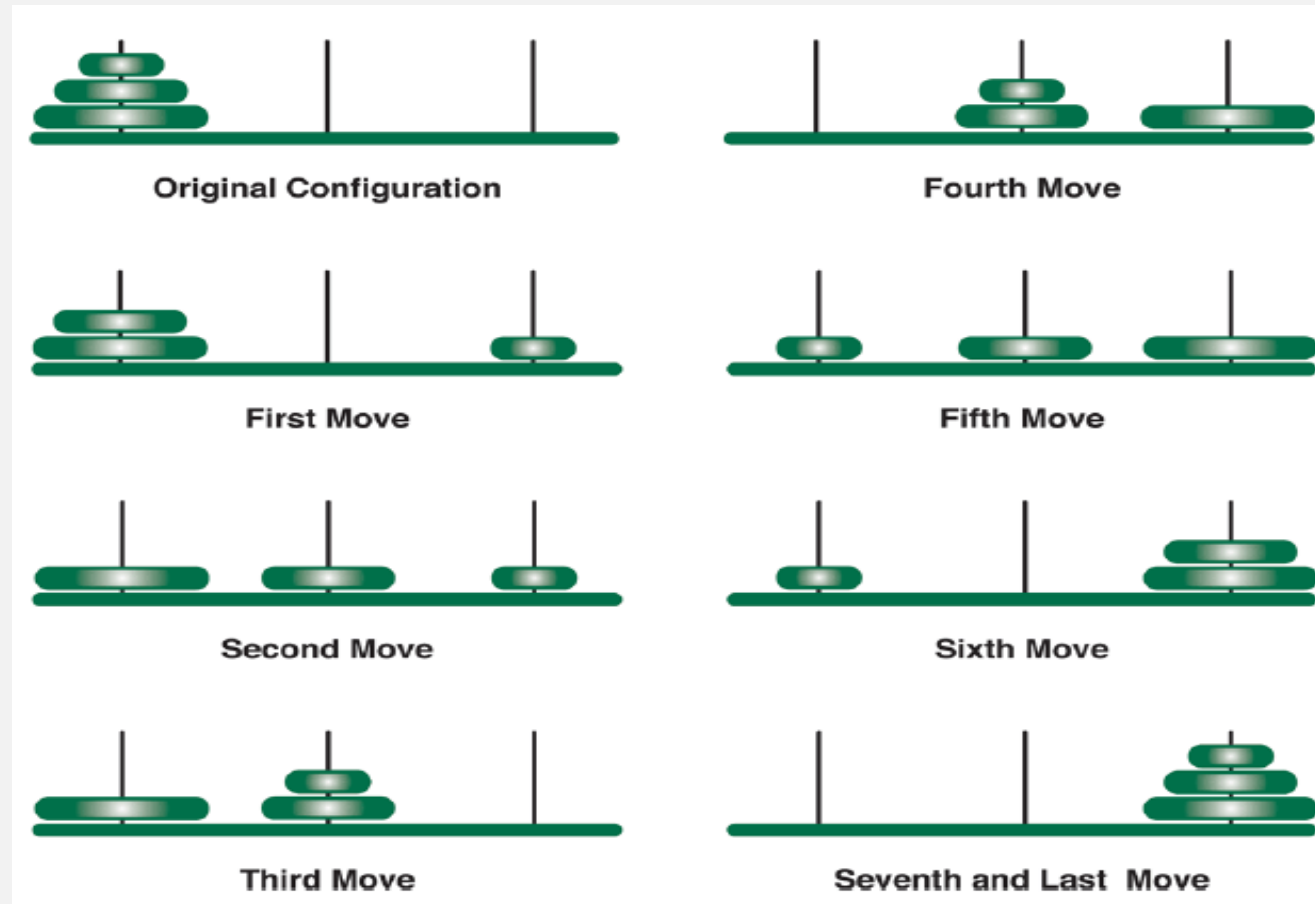
else return *BinRec*($\lfloor n/2 \rfloor$) + 1

Number of additions : $A(n) = A(\lfloor n/2 \rfloor) + 1$ for $n > 1$.
 $A(1) = 0$.

Algorithm analysis

Analysis of Recursive Algorithms

Example 3: The Tower of Hanoi



Algorithm analysis

Analysis of Recursive Algorithms

Example 3: The Tower of Hanoi

```
class Main {  
public static void main(String[] args) {  
TOH(3, 1,2,3);  
}  
public static void TOH(int n, int A, int B, int C){  
    if (n > 0){  
        TOH(n-1, A, C, B);  
        System.out.printf("Move a Disc from %d to %d %n",A, C);  
        TOH(n-1, B, A, C);  
    }  
}  
}
```

Algorithm analysis

Analysis of Recursive Algorithms

Example 3: The Tower of Hanoi

Number of moves : $M(n) = 2M(n - 1) + 1$ for $n > 1$,
 $M(1) = 1$.

$$\begin{aligned} M(n) &= 2M(n - 1) + 1 && \text{sub. } M(n - 1) = 2M(n - 2) + 1 \\ &= 2[2M(n - 2) + 1] + 1 = 2^2M(n - 2) + 2 + 1 && \text{sub. } M(n - 2) = 2M(n - 3) + 1 \\ &= 2^2[2M(n - 3) + 1] + 2 + 1 = 2^3M(n - 3) + 2^2 + 2 + 1. \end{aligned}$$

$$M(n) = 2^i M(n - i) + 2^{i-1} + 2^{i-2} + \dots + 2 + 1 = 2^i M(n - i) + 2^i - 1.$$

$$\begin{aligned} M(n) &= 2^{n-1}M(n - (n - 1)) + 2^{n-1} - 1 \\ &= 2^{n-1}M(1) + 2^{n-1} - 1 = 2^{n-1} + 2^{n-1} - 1 = 2^n - 1. \end{aligned}$$